

EC200U&EG91xU Series QuecOpen(SDK) **SDMMC Development Guide**

LTE Standard Module Series

Version: 1.1

Date: 2023-11-06

Status: Released



At Quectel, our aim is to provide timely and comprehensive services to our customers. If you require any assistance, please contact our headquarters:

Quectel Wireless Solutions Co., Ltd.

Building 5, Shanghai Business Park Phase III (Area B), No.1016 Tianlin Road, Minhang District, Shanghai 200233, China

Tel: +86 21 5108 6236

Email: info@quectel.com

Or our local offices. For more information, please visit:

<http://www.quectel.com/support/sales.htm>.

For technical support, or to report documentation errors, please visit:

<http://www.quectel.com/support/technical.htm>.

Or email us at: support@quectel.com.

Legal Notices

We offer information as a service to you. The provided information is based on your requirements and we make every effort to ensure its quality. You agree that you are responsible for using independent analysis and evaluation in designing intended products, and we provide reference designs for illustrative purposes only. Before using any hardware, software or service guided by this document, please read this notice carefully. Even though we employ commercially reasonable efforts to provide the best possible experience, you hereby acknowledge and agree that this document and related services hereunder are provided to you on an “as available” basis. We may revise or restate this document from time to time at our sole discretion without any prior notice to you.

Use and Disclosure Restrictions

License Agreements

Documents and information provided by us shall be kept confidential, unless specific permission is granted. They shall not be accessed or used for any purpose except as expressly provided herein.

Copyright

Our and third-party products hereunder may contain copyrighted material. Such copyrighted material shall not be copied, reproduced, distributed, merged, published, translated, or modified without prior written consent. We and the third party have exclusive rights over copyrighted material. No license shall be granted or conveyed under any patents, copyrights, trademarks, or service mark rights. To avoid ambiguities, purchasing in any form cannot be deemed as granting a license other than the normal non-exclusive, royalty-free license to use the material. We reserve the right to take legal action for noncompliance with abovementioned requirements, unauthorized use, or other illegal or malicious use of the material.

Trademarks

Except as otherwise set forth herein, nothing in this document shall be construed as conferring any rights to use any trademark, trade name or name, abbreviation, or counterfeit product thereof owned by Quectel or any third party in advertising, publicity, or other aspects.

Third-Party Rights

This document may refer to hardware, software and/or documentation owned by one or more third parties ("third-party materials"). Use of such third-party materials shall be governed by all restrictions and obligations applicable thereto.

We make no warranty or representation, either express or implied, regarding the third-party materials, including but not limited to any implied or statutory, warranties of merchantability or fitness for a particular purpose, quiet enjoyment, system integration, information accuracy, and non-infringement of any third-party intellectual property rights with regard to the licensed technology or use thereof. Nothing herein constitutes a representation or warranty by us to either develop, enhance, modify, distribute, market, sell, offer for sale, or otherwise maintain production of any our products or any other hardware, software, device, tool, information, or product. We moreover disclaim any and all warranties arising from the course of dealing or usage of trade.

Privacy Policy

To implement module functionality, certain device data are uploaded to Quectel's or third-party's servers, including carriers, chipset suppliers or customer-designated servers. Quectel, strictly abiding by the relevant laws and regulations, shall retain, use, disclose or otherwise process relevant data for the purpose of performing the service only or as permitted by applicable laws. Before data interaction with third parties, please be informed of their privacy and data security policy.

Disclaimer

- a) We acknowledge no liability for any injury or damage arising from the reliance upon the information.
- b) We shall bear no liability resulting from any inaccuracies or omissions, or from the use of the information contained herein.
- c) While we have made every effort to ensure that the functions and features under development are free from errors, it is possible that they could contain errors, inaccuracies, and omissions. Unless otherwise provided by valid agreement, we make no warranties of any kind, either implied or express, and exclude all liability for any loss or damage suffered in connection with the use of features and functions under development, to the maximum extent permitted by law, regardless of whether such loss or damage may have been foreseeable.
- d) We are not responsible for the accessibility, safety, accuracy, availability, legality, or completeness of information, advertising, commercial offers, products, services, and materials on third-party websites and third-party resources.

Copyright © Quectel Wireless Solutions Co., Ltd. 2023. All rights reserved.

About the Document

Revision History

Version	Date	Author	Description
-	2021-12-07	Sum LI	Creation of the document
1.0	2022-01-04	Sum LI	First official release
1.1	2023-11-06	Sum LI	- Updated the document name based on the unified QuecOpen naming. - Added the applicable modules EG912U-GL and EG915U series. - Updated the document structure and divided APIs into driver-related APIs and filesystem-related APIs (Chapters 2.2 & 2.3) - Updated the SDMMC API error code enumeration (Chapter 2.2.2.1.1) - Added the following APIs: ql_sdmmc_send_write_prot() (Chapter 2.2.2.9) ql_sdmmc_register_power_reset_cb() (Chapter 2.2.2.10) ql_sdmmc_set_dev_cfg() (Chapter 2.2.2.11) ql_sdmmc_fcreate_file() (Chapter 2.3.2.13) ql_sdmmc_fat_lock() (Chapter 2.3.2.14) ql_sdmmc_fat_unlock() (Chapter 2.3.2.15) ql_sdmmc_fat_is_lock() (Chapter 2.3.2.16) - Updated the opt parameter description in ql_sdmmc_mkfs() and ql_sdmmc_mkfs_ex() (Chapters 2.3.2.3 & 2.3.2.7). - Added FM_FAT type to SDMMC formatting and the note about selecting the type of SDMMC formatting (Chapter 2.3.2.3). - Updated the log capture information of SDMMC debugging preparation (Chapter 3.2.1).

Contents

About the Document.....	3
Contents	4
Table Index.....	6
Figure Index	7
1 Introduction	8
1.1. Applicable Modules.....	8
2 SDMMC API.....	9
2.1. Header File.....	9
2.2. Driver Related API.....	9
2.2.1. API Overview.....	9
2.2.2. API Description	10
2.2.2.1. ql_sdmmc_open.....	10
2.2.2.1.1. ql_errcode_sdmmc_e	10
2.2.2.2. ql_sdmmc_close	12
2.2.2.3. ql_sdmmc_write	13
2.2.2.4. ql_sdmmc_read.....	13
2.2.2.5. ql_sdmmc_set_clk.....	14
2.2.2.6. ql_sdmmc_get_block_number	14
2.2.2.7. ql_sdmmc_get_hw_info	15
2.2.2.7.1. ql_sdmmc_hw_info_t.....	15
2.2.2.8. ql_sdmmc_sdio1_vdd_off	16
2.2.2.9. ql_sdmmc_send_write_prot.....	16
2.2.2.10. ql_sdmmc_register_power_reset_cb.....	17
2.2.2.10.1. ql_sdmmc_pw_rst_cb	17
2.2.2.11. ql_sdmmc_set_dev_cfg	18
2.2.2.11.1. ql_sdmmc_cfg_t.....	18
2.2.2.11.2. ql_sdmmc_dev_detect_e	19
2.2.2.11.3. ql_sdmmc_data_size_e	20
2.3. File System Related API	20
2.3.1. API Overview.....	20
2.3.2. API Description	21
2.3.2.1. ql_sdmmc_mount.....	21
2.3.2.2. ql_sdmmc_umount.....	22
2.3.2.3. ql_sdmmc_mkfs	22
2.3.2.4. ql_sdmmc_is_mount	23
2.3.2.5. ql_sdmmc_is_format.....	24
2.3.2.6. ql_sdmmc_fdisk_ex.....	24
2.3.2.6.1. ql_sdmmc_part_info_t.....	25
2.3.2.6.2. ql_sdmmc_partition_num_e.....	25
2.3.2.7. ql_sdmmc_mkfs_ex	26
2.3.2.8. ql_sdmmc_mount_ex.....	26

2.3.2.9.	ql_sdmmc_umount_ex	27
2.3.2.10.	ql_sdmmc_is_fdisk_ex	27
2.3.2.11.	ql_sdmmc_is_format_ex	28
2.3.2.12.	ql_sdmmc_is_mount_ex	28
2.3.2.13.	ql_sdmmc_fcreate_file	29
2.3.2.14.	ql_sdmmc_fat_lock	30
2.3.2.15.	ql_sdmmc_fat_unlock	30
2.3.2.16.	ql_sdmmc_fat_is_lock.....	31
3	Development Example and Debugging	32
3.1.	Development Example	32
3.1.1.	SDMMC Demo Introduction	32
3.1.2.	SDMMC Demo Auto-startup	34
3.2.	SDMMC Debugging	35
3.2.1.	Debugging Preparation	35
3.2.2.	SDMMC Feature Debugging.....	36
4	Appendix References	37

Table Index

Table 1: Applicable Modules 8

Table 2: Driver Related API Overview 9

Table 3: File System Related API Overview 20

Table 4: Related Documents 37

Table 5: Terms and Abbreviations 37

Figure Index

Figure 1: Enable and Disable Feature with Macro Definition	32
Figure 2: Entry Function ql_sdmmc_app_init()	33
Figure 3: SDMMC Demo ql_init_demo_thread()	34
Figure 4: LTE OPEN EVB	35
Figure 5: AP Log Information	36

1 Introduction

Quectel LTE Standard EC200U series and EG91xU family modules support QuecOpen® solution. QuecOpen® is an embedded development platform based on RTOS. It is intended to simplify the design and development of IoT applications. For more information on QuecOpen®, see **document [1]**.

This document is applicable to QuecOpen® solution based on SDK build environment and outlines the SDMMC application of the module, including SDMMC API, data structure, development examples and debugging methods.

1.1. Applicable Modules

Table 1: Applicable Modules

Module Family	Module
-	EC200U Series
EG91xU	EG912U-GL
	EG915U Series

2 SDMMC API

2.1. Header File

ql_sdmmc.h, the header file of SDMMC API, is in the *components\ql-kernel\inc* directory. Unless otherwise specified, all header files mentioned in this document are in this directory.

2.2. Driver Related API

2.2.1. API Overview

Table 2: Driver Related API Overview

Function	Description
<i>ql_sdmmc_open()</i>	Initializes SDMMC device without mounting it to the file system.
<i>ql_sdmmc_close()</i>	De-initializes SDMMC device.
<i>ql_sdmmc_write()</i>	Writes the data with specified bytes to the specified sector of SDMMC.
<i>ql_sdmmc_read()</i>	Reads the data with specified bytes from the specified sector of SDMMC.
<i>ql_sdmmc_set_clk()</i>	Sets the clock frequency of SDMMC.
<i>ql_sdmmc_get_block_number()</i>	Gets the total number of SDMMC sectors.
<i>ql_sdmmc_get_hw_info()</i>	Gets the hardware information of SDMMC.
<i>ql_sdmmc_sdio1_vdd_off()</i>	Powers off the SDMMC communication pin.
<i>ql_sdmmc_send_write_prot</i>	Gets the write protection status of a sector.
<i>ql_sdmmc_register_power_reset_cb</i>	Registers an operation function to reset the SDMMC device through power on/off.
<i>ql_sdmmc_set_dev_cfg</i>	Sets the initialization parameters of the SDMMC device.

2.2.2. API Description

2.2.2.1. ql_sdmmc_open

This function initializes SDMMC device without mounting it to the file system. If the device is mounted to file system, it will be unmounted after this function is called.

It is only applicable to the scenario where SDMMC is operated through the driver layer of SDMMC device rather than the file system.

● Prototype

```
ql_errcode_sdmmc_e ql_sdmmc_open(void)
```

● Parameter

None

● Return Value

See **Chapter 2.2.2.1.1** for details.

2.2.2.1.1. ql_errcode_sdmmc_e

The enumeration of SDMMC API error codes:

```
typedef enum
{
    QL_SDMMC_SUCCESS                = QL_SUCCESS,
    QL_SDMMC_INVALID_PARAM_ERR      = 1|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_ADDR_NULL_ERR          = 3|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_INIT_ERR               = 4|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_MOUNT_ERR              = 5|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_MKFS_ERR               = 10|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_SET_CLK_ERR            = 11|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_WRITE_ERR              = 12|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_READ_ERR              = 13|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_DET_ERR               = 14|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_MEM_ILLEGAL_ERR        = 20|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_ADDR_ALIGNED_ERR       = 21|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_SIZE_ALIGNED_ERR       = 22|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_BLOCK_NUM_OVER_ERR     = 23|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_LOCK_ERR              = 24|QL_SDMMC_ERRCODE_BASE,
    QL_SDMMC_CREATE_ERR            = 30|QL_SDMMC_ERRCODE_BASE,
```

```

QL_SDMMC_OPEN_ERR           = 31|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_CLOSE_ERR          = 32|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_OPERATION_NOT_ALLOWED = 33|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_FAT_LOCK_ERR       = 34|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_FAT_CREATE_FILE_ERR = 35|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_MULTI_PARTITON_NUM_ERR = 40|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_MULTI_PARTITON_FDISK_ERR = 41|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_UMOUNT_ERR         = 42|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_MOUNT_ALL_ERR      = 43|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_UMOUNT_ALL_ERR     = 44|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_MKFS_ALL_ERR       = 45|QL_SDMMC_ERRCODE_BASE,
QL_SDMMC_MULTI_PARTITON_OTHER_ERR = 46|QL_SDMMC_ERRCODE_BASE,
} ql_errcode_sdmmc_e
    
```

● Member

Member	Description
<i>QL_SDMMC_SUCCESS</i>	Successful execution.
<i>QL_SDMMC_INVALID_PARAM_ERR</i>	Invalid parameter(s).
<i>QL_SDMMC_ADDR_NULL_ERR</i>	Null pointer address.
<i>QL_SDMMC_INIT_ERR</i>	Failed to initialize SDMMC device and mount it to the file system.
<i>QL_SDMMC_MOUNT_ERR</i>	Failed to mount SDMMC.
<i>QL_SDMMC_MKFS_ERR</i>	Failed to format SDMMC.
<i>QL_SDMMC_SET_CLK_ERR</i>	Failed to set clock frequency of SDMMC.
<i>QL_SDMMC_WRITE_ERR</i>	Failed to write data to SDMMC.
<i>QL_SDMMC_READ_ERR</i>	Failed to read data from SDMMC.
<i>QL_SDMMC_DET_ERR</i>	Failed to register hot plug interrupt.
<i>QL_SDMMC_MEM_ILLEGAL_ERR</i>	Illegal memory address.
<i>QL_SDMMC_ADDR_ALIGNED_ERR</i>	Memory address is not 32-byte aligned.
<i>QL_SDMMC_SIZE_ALIGNED_ERR</i>	Read and written data are not 512-byte aligned.
<i>QL_SDMMC_BLOCK_NUM_OVER_ERR</i>	Read and written sector number is out of range.
<i>QL_SDMMC_LOCK_ERR</i>	Failed to lock the mutex.
<i>QL_SDMMC_CREATE_ERR</i>	Failed to create a SDMMC handle.

<code>QL_SDMMC_OPEN_ERR</code>	Failed to initialize SDMMC device.
<code>QL_SDMMC_CLOSE_ERR</code>	Failed to de-initialize SDMMC device.
<code>QL_SDMMC_OPERATION_NOT_ALLOWED</code>	Illegal operation.
<code>QL_SDMMC_FAT_LOCK_ERR</code>	Failed to lock/unlock the FAT table and directory entries.
<code>QL_SDMMC_FAT_CREATE_FILE_ERR</code>	Failed to create a specified size file.
<code>QL_SDMMC_MULTI_PARTITON_NUM_ERR</code>	Multi-partition number error.
<code>QL_SDMMC_MULTI_PARTITON_FDISK_ERR</code>	Failed to partition.
<code>QL_SDMMC_UMOUNT_ERR</code>	Failed to unmount.
<code>QL_SDMMC_MOUNT_ALL_ERR</code>	Failed to mount all partitions.
<code>QL_SDMMC_UMOUNT_ALL_ERR</code>	Failed to unmount all partitions.
<code>QL_SDMMC_MKFS_ALL_ERR</code>	Failed to format all partitions.
<code>QL_SDMMC_MULTI_PARTITON_OTHER_ERR</code>	Other multi-partition operation error.

2.2.2.2. `ql_sdmmc_close`

This function de-initializes the SDMMC device that has been initialized through `ql_sdmmc_open()`. If the is mounted to file system, it will be unmounted after this function is called.

It is only applicable to the scenario where SDMMC is operated through the driver layer of SDMMC device rather than the file system.

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_close(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.2.2.3. ql_sdmmc_write

This function writes the data with specified bytes to the specified sector of SDMMC.

This function is only applicable to scenario where SDMMC is operated directly through the driver layer of the SDMMC device rather than the file system. Otherwise the partition information recorded in the SDMMC device may be destroyed. Once the partition is destroyed, the device must be formatted with `ql_sdmmc_mkfs()` before it is mounted to file system.

● Prototype

```
ql_errcode_sdmmc_e ql_sdmmc_write(uint32_t block_number, const void *buffer, uint32_t size)
```

● Parameter

block_number:

[In] Sector number of the data to be written.

buffer:

[In] Memory address where the data to be written is stored. It cannot be a NULL pointer and must be 32-byte aligned .

size:

[In] Size of the data to be written. It must be 512-byte aligned and one sector is 512 bytes.

● Return Value

See **Chapter 2.2.2.1.1** for details.

2.2.2.4. ql_sdmmc_read

This function reads the data with specified bytes from the specified sector of SDMMC. It is only applicable to the scenario where SDMMC is operated directly through the driver layer of the SDMMC device rather than the file system.

● Prototype

```
ql_errcode_sdmmc_e ql_sdmmc_read(uint32_t block_number, void *buffer, uint32_t size)
```

● Parameter

block_number:

[In] Sector number of data to be read.

buffer:

[In] Memory address where the data to be read is stored. It cannot be a NULL pointer and must be 32-byte aligned.

size:

[In] Size of the data to be read. It must be 512-byte aligned and one sector is 512 bytes.

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.2.2.5. ql_sdmmc_set_clk

This function sets the clock frequency of SDMMC. By default, the clock frequency of SDSC device is 25 MHz and that of SDHC/SDXC/eMMC device is 50 MHz.

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_set_clk(uint32_t freq)
```

- **Parameter**

freq:

[In] Clock frequency to be set. Range: 400000–50000000. Unit: Hz.

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.2.2.6. ql_sdmmc_get_block_number

This function gets the total number of SDMMC sectors.

- **Prototype**

```
uint32_t ql_sdmmc_get_block_number(void)
```

- **Parameter**

None

- **Return Value**

Total number of SDMMC sectors.

2.2.2.7. ql_sdmmc_get_hw_info

This function gets the hardware information of SDMMC.

● Prototype

```
ql_errcode_sdmmc_e ql_sdmmc_get_hw_info(ql_sdmmc_hw_info_t *info)
```

● Parameter

info:

[Out] Obtained data. See **Chapter 2.2.2.7.1** for details.

● Return Value

See **Chapter 2.2.2.1.1** for details.

2.2.2.7.1. ql_sdmmc_hw_info_t

The structure of the hardware information of SDMMC:

```
typedef struct
{
    uint8_t mid;
    uint8_t pnm[6];
    uint8_t psn[4];
    uint32_t blknum;
    uint32_t blksize;
} ql_sdmmc_hw_info_t
```

● Parameter

Type	Parameter	Description
uint8_t	<i>mid</i>	Manufacturer ID.
uint8_t	<i>pnm</i>	Product ID. Maximum length: 6 bytes.
uint8_t	<i>psn</i>	Product serial number. Maximum length: 4 bytes.
uint32_t	<i>blknum</i>	Number of device sectors.
uint32_t	<i>blksize</i>	Size of one device sector.

2.2.2.8. ql_sdmmc_sdio1_vdd_off

This function powers off the SDMMC communication pin. Please think carefully and master the precautions from *ql_sdmmc.h* before using this function.

- **Prototype**

```
void ql_sdmmc_sdio1_vdd_off(void)
```

- **Parameter**

None

- **Return Value**

None

2.2.2.9. ql_sdmmc_send_write_prot

This function gets the write protection status of a sector.

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_send_write_prot(uint32_t addr, char *prot_data)
```

- **Parameter**

addr:

[In] The address for which the write protection status needs to be obtained.

prot_data:

[Out] The obtained write protection status. The status is 4 bytes consisting of 32 write protection bits (representing 32 write protection groups).

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.2.2.10. ql_sdmmc_register_power_reset_cb

This function registers an operation function to reset the SDMMC device through power on/off. It powers off the SD card/eMMC in case of SDMMC exceptions, then initiates a reinitialization process to attempt automatic recovery. If SDMMC uses GPIO to power on/off the SD card or eMMC, the power-off and power-on process should be registered. If the power control is managed through the SDMMC voltage domain, registration is not required.

● Prototype

```
ql_errcode_sdmmc_e ql_sdmmc_register_power_reset_cb(ql_sdmmc_pw_rst_cb cb, bool
retry_enable)
```

● Parameter

cb:

[In] Callback function for resetting the SDMMC device through powering on/off. See **Chapter 2.2.2.10.1** for details.

retry_enable:

[In] Whether to enable the operation of resetting the SDMMC by powering on/off to automatically recover the SDMMC device when an exception occurs.

TRUE Enable

FALSE Disable

● Return Value

See **Chapter 2.2.2.1.1** for details.

2.2.2.10.1.ql_sdmmc_pw_rst_cb

This callback function resets the SDMMC device through powering on/off.

● Prototype

```
typedef void (*ql_sdmmc_pw_rst_cb)(void)
```

● Parameter

None

● Return Value

None

2.2.2.11. ql_sdmmc_set_dev_cfg

This function configures the initialization parameters of the SDMMC device.

● Prototype

```
ql_errcode_sdmmc_e ql_sdmmc_set_dev_cfg(ql_sdmmc_cfg_t cfg)
```

● Parameter

cfg:

[In] Configuration parameters. See **Chapter 2.2.2.11.1** for details.

● Return Value

See **Chapter 2.2.2.1.1** for details.

2.2.2.11.1. ql_sdmmc_cfg_t

The structure of SDMMC configuration parameters:

```
typedef struct
{
    ql_sdmmc_dev_detect_e dev;
    uint32_t sd_mv;
    uint32_t emmc_mv;
    uint8_t sd_drv;
    uint8_t emmc_drv;
    ql_sdmmc_data_size_e data_size;
} ql_sdmmc_cfg_t
```

● Parameter

Type	Parameter	Description
<i>ql_sdmmc_dev_detect_e</i>	<i>dev</i>	Device type. See Chapter 2.2.2.11.2 for details.
uint32_t	<i>sd_mv</i>	Voltage domain for SD card operation. Range: 0–3200. Unit: V. Default value: 3.2 V. If it's 0 or less than 1.7 V, the default value is used.
uint32_t	<i>emmc_mv</i>	Voltage domain for eMMC operation. Range: 0–3200; Unit: V. Default value: 1.8 V. If it's 0 or less than 1.7 V, the default value is used.
uint8_t	<i>sd_drv</i>	Drive capability for all SD card pins. It is used when the SDMMC pin drive capability is the

		same. Values and their corresponding drive currents are specified as follows: 0: Driven strength 3 mA 1: Driven strength 6 mA 2: Driven strength 9 mA 3: Driven strength 12 mA 4: Driven strength 15 mA 5: Driven strength 18 mA 6: Driven strength 21 mA 7: Driven strength 24 mA 8: Driven strength 27 mA 9: Driven strength 30 mA 10: Driven strength 33 mA 11: Driven strength 36 mA 12: Driven strength 39 mA 13: Driven strength 42 mA 14: Driven strength 45 mA 15: Driven strength 48 mA
uint8_t	emmc_drv	Drive capability for all eMMC pins. It is used when the SDMMC pin drive capability is the same. Values and their corresponding drive currents are same as <i>sd_drv</i>.
ql_sdmmc_data_size_e	data_size	Data cable width of SDMMC. See Chapter 2.2.2.11.3 for details.

2.2.2.11.2.ql_sdmmc_dev_detect_e

The enumeration of SDMMC initialization device detection types:

```
typedef enum
{
    QL_SDMMC_ = 0,
    QL_SDMMC_SD_CARD,
    QL_SDMMC__ONLY,
    QL_SDMMC_SD_CARD_ONLY
} ql_sdmmc_dev_detect_e
```

- Member

Member	Description
<i>QL_SDMMC_</i>	Initialize with eMMC type process first, and if it fails, initialize with the SD card type process.
<i>QL_SDMMC_SD_CARD</i>	Initialize with SD card type process first, and if it fails, initialize with the eMMC type process.
<i>QL_SDMMC_ONLY</i>	Initialize with eMMC type process only.
<i>QL_SDMMC_SD_CARD_ONLY</i>	Initialize with SD card type process only.

2.2.2.11.3.ql_sdmmc_data_size_e

The enumeration of SDMMC data cable width:

```
typedef enum
{
    QL_SDMMC_DATA_SIZE_4BIT = 0,
    QL_SDMMC_DATA_SIZE_1BIT,
} ql_sdmmc_data_size_e
```

- Member

Member	Description
<i>QL_SDMMC_DATA_SIZE_4BIT</i>	SDMMC data cable width is 4 bits: DATA0–DATA3.
<i>QL_SDMMC_DATA_SIZE_1BIT</i>	SDMMC data cable width is 1 bit: DATA0.

2.3. File System Related API

2.3.1. API Overview

Table 3: File System Related API Overview

Function	Description
<i>ql_sdmmc_mount()</i>	Initializes SDMMC device and mounts it to the file system.
<i>ql_sdmmc_umount()</i>	De-initializes SDMMC device and unmounts it from the file system.
<i>ql_sdmmc_mkfs()</i>	Formats SDMMC device.

<i>ql_sdmmc_is_mount()</i>	Determines whether the SDMMC device is mounted to the file system.
<i>ql_sdmmc_is_format()</i>	Determines whether the SDMMC device is formatted.
<i>ql_sdmmc_fdisk_ex()</i>	Partitions the SDMMC device.
<i>ql_sdmmc_mkfs_ex()</i>	Formats the specified partition of SDMMC device.
<i>ql_sdmmc_mount_ex()</i>	Initializes the specified partition and mounts it to the file system.
<i>ql_sdmmc_umount_ex()</i>	De-initializes the specified partition and unmounts it from the file system.
<i>ql_sdmmc_is_fdisk_ex()</i>	Determines whether the SDMMC device is partitioned.
<i>ql_sdmmc_is_format_ex()</i>	Determines whether the partition is formatted.
<i>ql_sdmmc_is_mount_ex()</i>	Determines whether the partition is mounted.
<i>ql_sdmmc_fcreate_file</i>	Creates a file of a specified size with random content.
<i>ql_sdmmc_fat_lock</i>	Locks the FAT table and directory entries.
<i>ql_sdmmc_fat_unlock</i>	Unlocks the FAT table and directory entries.
<i>ql_sdmmc_fat_is_lock</i>	Checks if the FAT table and directory entries of the file system mounted on the SDMMC device are locked.

2.3.2. API Description

2.3.2.1. ql_sdmmc_mount

This function initializes SDMMC device and mounts it to the file system. It is only applicable to the scenario where SDMMC is operated through the file system.

When the SDMMC device is not partitioned, this function can be called to initialize the SDMMC device and mount it to the file system. When the SDMMC device is partitioned, it is recommended to call *ql_sdmmc_mount_ex()* to initialize the partition and mount it to the file system; if *ql_sdmmc_mount()* rather than *ql_sdmmc_mount_ex()* is called at this time, the first partition is initialized and mounted to the file system by default.

● Prototype

```
ql_errcode_sdmmc_e ql_sdmmc_mount(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.3.2.2. `ql_sdmmc_umount`

This function de-initializes the SDMMC device that has been initialized and mounted through `ql_sdmmc_mount()` and unmounts it from the file system. It is only applicable to the scenario where SDMMC is operated through the file system.

When the SDMMC device is not partitioned, this function can be called to de-initialize the SDMMC device and unmount it from the file system. When the SDMMC device is partitioned, it is recommended to call `ql_sdmmc_umount_ex()` to de-initialize the partition and unmount it from the file system; if `ql_sdmmc_umount()` is called rather than `ql_sdmmc_umount_ex()`, the first partition is de-initialized and unmounted from the file system by default.

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_umount(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.3.2.3. `ql_sdmmc_mkfs`

This function formats SDMMC device. The SDMMC device will be NULL when the file system re-reads it after the function is executed successfully. This function is only applicable to the scenario where the SDMMC is operated through the file system.

If this function is called to format the partitioned SDMMC device, all partitions will be merged before they are formatted, that is, the SDMMC device will be restored to an unpartitioned state. Therefore, it is recommended to call `ql_sdmmc_mkfs_ex()` to format the specified partition.

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_mkfs(uint8_t opt)
```

- **Parameter**

opt:

[In] The target formatting.

0x01	FM_FAT (FAT12/FAT16)
0x02	FM_FAT32

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

NOTE

You need to select the formatting type according to the number of sectors in the SDMMC device. The correspondence between the sector number and the formatting types defined by the file system is as follows:

1024–8220 sectors: Only support FM_FAT (FAT12) formatting.

8229–4190430 sectors: You can choose FM_FAT (FAT16) formatting.

66130–67048580 sectors: You can choose FM_FAT32 formatting.

Therefore:

If the sector number is less than 66130, it can only be formatted as FM_FAT.

If the sector number is between 66130 and 4190430, it can be formatted as either FM_FAT32 or FM_FAT.

If the sector number is greater than or equal to 4190431, it can only be formatted as FM_FAT32.

2.3.2.4. ql_sdmmc_is_mount

This function determines whether the SDMMC device is mounted to the file system.

- **Prototype**

```
bool ql_sdmmc_is_mount(void)
```

- **Parameter**

None

- **Return Value**

TRUE SDMMC device is mounted to the file system.

FALSE SDMMC device is not mounted to the file system.

2.3.2.5. ql_sdmmc_is_format

This function determines whether the SDMMC device is formatted.

- **Prototype**

```
bool ql_sdmmc_is_format(void)
```

- **Parameter**

None

- **Return Value**

TRUE SDMMC device is formatted.

FALSE SDMMC device is not formatted.

2.3.2.6. ql_sdmmc_fdisk_ex

This function partitions the SDMMC device. Only 2 partitions are currently supported.

After the size of the first partition is set, the remaining memory of the SDMMC device will be automatically allocated to the second partition. Because the file system partition must contain at least 1024 sectors, the size of each partition cannot be less than the total size of 1024 sectors (1024 × 512 bytes).

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_fdisk_ex(ql_sdmmc_part_info_t part_info)
```

- **Parameter**

part_info:

[In] Partition information. See **Chapter 2.3.2.6.1** for details.

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.3.2.6.1. ql_sdmmc_part_info_t

The structure of SDMMC partition information:

```
typedef struct
{
    ql_sdmmc_partition_num_e partnum;
    int64 partsize;
} ql_sdmmc_part_info_t
```

● Parameter

Type	Parameter	Description
<i>ql_sdmmc_partition_num_e</i>	<i>partnum</i>	Partition number. See Chapter 2.3.2.6.2 for details.
int64	<i>partsize</i>	Partition size. Unit: MB.

2.3.2.6.2. ql_sdmmc_partition_num_e

The enumeration of SDMMC partition number:

```
typedef enum
{
    QL_SDMMC_PARTITION_NUM_1    = 1,
    QL_SDMMC_PARTITION_NUM_2    = 2,
    QL_SDMMC_PARTITION_NUM_MAX,
    QL_SDMMC_PARTITION_NUM_ALL = 0xff
} ql_sdmmc_partition_num_e
```

● Member

Member	Description
<i>QL_SDMMC_PARTITION_NUM_1</i>	First partition.
<i>QL_SDMMC_PARTITION_NUM_2</i>	Second partition.
<i>QL_SDMMC_PARTITION_NUM_MAX</i>	Number indicating the largest partition.
<i>QL_SDMMC_PARTITION_NUM_ALL</i>	Numbers indicating all available partitions.

2.3.2.7. ql_sdmmc_mkfs_ex

This function formats the specified partition of SDMMC devices. Only 2 partitions are currently supported.

This function can automatically adjust the formatting type according to the sector number of the partitions. If the total number of sectors is below 66130, the formatting type is FM_FAT. If the total number of sectors is greater than or equal to 66130, the type is FM_FAT32.

● Prototype

```
ql_errcode_sdmmc_e ql_sdmmc_mkfs_ex(ql_sdmmc_partition_num_e part,uint8_t opt)
```

● Parameter

part:

[In] Partition number.

QL_SDMMC_PARTITION_NUM_1: First partition

QL_SDMMC_PARTITION_NUM_2: Second partition

QL_SDMMC_PARTITION_NUM_ALL: All partitions

opt:

[In] Type of partition formatting.

0x01 FM_FAT (FAT12/FAT16)

0x02 FM_FAT32

● Return Value

See **Chapter 2.2.2.1.1** for details.

2.3.2.8. ql_sdmmc_mount_ex

This function initializes the specified partition and mounts it to the file system. Only 2 partitions are currently supported.

● Prototype

```
ql_errcode_sdmmc_e ql_sdmmc_mount_ex(ql_sdmmc_partition_num_e part)
```

- **Parameter**

part:

[In] Partition number.

QL_SDMMC_PARTITION_NUM_1: First partition

QL_SDMMC_PARTITION_NUM_2: Second partition

QL_SDMMC_PARTITION_NUM_ALL: All partitions

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.3.2.9. ql_sdmmc_umount_ex

This function de-initializes the specified partition and unmounts it from the file system. Only 2 partitions are currently supported.

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_umount_ex(ql_sdmmc_partition_num_e part)
```

- **Parameter**

part:

[In] Partition number.

QL_SDMMC_PARTITION_NUM_1: First partition

QL_SDMMC_PARTITION_NUM_2: Second partition

QL_SDMMC_PARTITION_NUM_ALL: All partitions

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.3.2.10. ql_sdmmc_is_fdisk_ex

This function determines whether the SDMMC device is partitioned.

- **Prototype**

```
bool ql_sdmmc_is_fdisk_ex(void)
```

- **Parameter**

None

- **Return Value**

TRUE SDMMC device is partitioned.

FALSE SDMMC device is not partitioned.

2.3.2.11. ql_sdmmc_is_format_ex

This function determines whether the partition is formatted.

- **Prototype**

```
bool ql_sdmmc_is_format_ex(ql_sdmmc_partition_num_e part)
```

- **Parameter**

part:

[In] Partition number.

QL_SDMMC_PARTITION_NUM_1: First partition

QL_SDMMC_PARTITION_NUM_2: Second partition

QL_SDMMC_PARTITION_NUM_ALL: All partitions

- **Return Value**

TRUE Partition is formatted.

FALSE Partition is not formatted.

2.3.2.12. ql_sdmmc_is_mount_ex

This function determines whether the SDMMC device partition is mounted.

- **Prototype**

```
bool ql_sdmmc_is_mount_ex(ql_sdmmc_partition_num_e part)
```

- **Parameter**

part:

[In] Partition number.

QL_SDMMC_PARTITION_NUM_1: First partition

QL_SDMMC_PARTITION_NUM_2: Second partition

QL_SDMMC_PARTITION_NUM_ALL: All partitions

- **Return Value**

TRUE SDMMC device partition is mounted.

FALSE SDMMC device partition is not mounted.

2.3.2.13. *ql_sdmmc_fcreate_file*

This function creates a file of a specified size with random content.

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_fcreate_file(const char *file_name, const char *mode, size_t size)
```

- **Parameter**

file_name:

[In] File name (with path). The length should not exceed 132 bytes, following the standard format "disk name:/path name/file name".

SD SD card partition

mode:

[In] File opening mode. It is consistent with the C standard library, commonly used modes like "wb+".

size:

[In] Size of the file to be created, cannot be 0. Unit: bytes.

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.3.2.14. ql_sdmmc_fat_lock

This function locks the FAT table and directory entries. After they are locked, the module cannot extend the FAT table or update directory entries when operating the file system. If an operation requires extending the FAT table or updating directory entries, an error will be returned.

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_fat_lock(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.3.2.15. ql_sdmmc_fat_unlock

This function unlocks the FAT table and directory entries. The file system recovers after being unlocked.

- **Prototype**

```
ql_errcode_sdmmc_e ql_sdmmc_fat_unlock(void)
```

- **Parameter**

None

- **Return Value**

See **Chapter 2.2.2.1.1** for details.

2.3.2.16. ql_sdmmc_fat_is_lock

This function checks if the FAT table and directory entries of the file system mounted on the SDMMC device are locked.

- **Prototype**

```
bool ql_sdmmc_fat_is_lock(void)
```

- **Parameter**

None

- **Return Value**

TRUE Locked

FALSE Not locked

3 Development Example and Debugging

This chapter outlines how to use SDMMC API for SDMMC development and simple debugging.

3.1. Development Example

3.1.1. SDMMC Demo Introduction

ql_sdmmc_demo.c, the SDMMC example file provided in the QuecOpen SDK codes of EC200U series and EG91xU family modules, is in the *lcomponents\ql-application\sdmmc* directory.

The three functional blocks in *ql_sdmmc_demo.c* are:

- 1) Mount, read and write SDMMC device with the file system
- 2) Open, close, read and write SDMMC device without the file system
- 3) Hot swap routine

Enable and disable each feature by using macro definition. 1) and 2) cannot be enabled at the same time, as shown in the figure below:

```
#define QL_SDMMC_FS_TEST           0 //Start the file system API test.
#define QL_SDMMC_MUTIL_PARTITION_TEST 0 //Multi-partition function.

#define QL_SDMMC_EVENT_PLUGOUT     0
#define QL_SDMMC_EVENT_INSERT     1
#define QL_SDMMC_DET_TEST         1

/******/
#define QL_SDMMC_ONLY_USE_DRIVER 0 //0-Use the file system
                                1-Only use SDMMC driver layer
/******/
```

Figure 1: Enable and Disable Feature with Macro Definition

The entry function is `ql_sdmmc_app_init()`, as shown in the figure below:

```

1 void ql_sdmmc_app_init(void)
2 {
3     QLOSStatus err = QL_SDMMC_SUCCESS;
4
5     /*sd pin init*/
6     ql_sdmmc_pin_init();
7
8     #if QL_SDMMC_ONLY_USE_DRIVER == 0
9         err = ql_sdmmc_mount_demo();
10        if(err != QL_SDMMC_SUCCESS)
11        {
12            QL_SDMMC_DEMO_LOG("sdmmc mount demo fail err = %d", err);
13        }
14    #endif
15
16    #ifdef QL_SDMMC_DET_TEST
17        err = ql_rtos_task_create(&ql_sdmmc_det_task, QL_SDMMC_TASK_STACK_SIZE, QL_SDMMC_TASK_Prio, "sdDEMO_det", ql_sdmmc_demo_det_thread, NL
18        if (err != QL_OSI_SUCCESS)
19        {
20            QL_SDMMC_DEMO_LOG("creat sd task fail err = %d", err);
21        }
22
23        err = ql_rtos_timer_create(&ql_sdmmc_det_debounce_timer, ql_sdmmc_det_task, ql_sdmmc_det_callback, NULL);
24        if(err != QL_OSI_SUCCESS)
25        {
26            QL_SDMMC_DEMO_LOG("creat timer task fail err = %d", err);
27        }
28    #endif
29
30    #if QL_SDMMC_ONLY_USE_DRIVER
31        ql_task_t sdmmc_task = NULL;
32        err = ql_rtos_task_create(&sdmmc_task, QL_SDMMC_TASK_STACK_SIZE, QL_SDMMC_TASK_Prio, "sdDEMO", ql_sdmmc_demo_thread, NULL, QL_SDMMC_TA
33        if (err != QL_OSI_SUCCESS)
34        {
35            QL_SDMMC_DEMO_LOG("creat sd task failed err = %d", err);
36        }
37    #else
38        #if QL_SDMMC_FS_TEST
39        #ifdef CONFIG_QUEC_PROJECT_FEATURE_FILE
40            ql_task_t sdmmc_task_fs = NULL;
41            err = ql_rtos_task_create(&sdmmc_task_fs, QL_SDMMC_TASK_STACK_SIZE, QL_SDMMC_TASK_Prio, "sdDEMO_fs", ql_sdmmc_demo_fs_thread, NULL, QL
42            if(err != QL_OSI_SUCCESS)
43            {
44                QL_SDMMC_DEMO_LOG("creat sd task fs failed err = %d", err);
45            }
46        #endif
47    #endif
48    #endif
49    } « end ql_sdmmc_app_init »

```

Figure 2: Entry Function `ql_sdmmc_app_init()`

This demo initializes the pins occupied by the SDMMC feature and according to the opened macro, initializes the SDMMC device and mounts it to the file system. Then it reads the total number of sectors of the SDMMC device and reads (writes) the data of the specified byte size from (to) the specified sector.

It should be noted that when initializing the pins occupied by the SDMMC function, the occupied GPIO pins cannot be used as general-purpose I/O ports. Otherwise, the initialization of the SDMMC device will fail. The number indicating the GPIO pin occupied by the SDMMC function can be obtained in `ql_sdmmc_pin_init()`, as shown in the following figure:

```

1 void ql_sdmmc_pin_init(void)
2 {
3     #ifdef QL_SDMMC_DET_TEST
4         ql_pin_set_func(QL_PIN_SDMMC_PIN_DET, QL_PIN_SDMMC_MODE_FUNC_GPIO); //Pin reuse
5         ql_gpio_deinit(GPIO_15);
6     #endif
7
8     ql_pin_set_func(QL_PIN_SDMMC_CMD, QL_PIN_SDMMC_MODE_FUNC); //Pin reuse
9     ql_pin_set_func(QL_PIN_SDMMC_DATA_0, QL_PIN_SDMMC_MODE_FUNC); //Pin reuse
10    ql_pin_set_func(QL_PIN_SDMMC_DATA_1, QL_PIN_SDMMC_MODE_FUNC); //Pin reuse
11    ql_pin_set_func(QL_PIN_SDMMC_DATA_2, QL_PIN_SDMMC_MODE_FUNC); //Pin reuse
12    ql_pin_set_func(QL_PIN_SDMMC_DATA_3, QL_PIN_SDMMC_MODE_FUNC); //Pin reuse
13    ql_pin_set_func(QL_PIN_SDMMC_CLK, QL_PIN_SDMMC_MODE_FUNC); //Pin reuse
14 }

```

NOTE

See **document [2]** for details on SDMMC device reading and writing through the file system.

3.1.2. SDMMC Demo Auto-startup

By default, SDMMC Demo is not enabled in `ql_init_demo_thread()`. Uncomment the code as shown in the following figure when you run a test.

```

static void ql_init_demo_thread(void *param)
{
    QL_INIT_LOG("init demo thread enter, param 0x%x", param);

    /*Caution:If the macro of secure boot and the function are opened
    the secret key cannot be changed forever*/
    #ifdef QL_APP_FEATURE_SECURE_BOOT
        //ql_dev_enable_secure_boot();
    #endif

    /*Caution: GPIO pin must be initialized here, otherwise the
    ql_pin_cfg_init();

    #if 0
        ql_gpio_app_init();
        ql_gpioint_app_init();
    #endif

    #ifdef QL_APP_FEATURE_LEDCFG
        ql_ledcfg_app_init();
    #endif

    #ifdef QL_APP_FEATURE_AUDIO
        //ql_audio_app_init();
    #endif

    #ifdef QL_APP_FEATURE_TTS
        //ql_tts_task_init();
    #endif

    #ifdef QL_APP_FEATURE_LCD
        //ql_lcd_app_init();
    #endif
    #ifdef QL_APP_FEATURE_LVGL
        //ql_lvgl_app_init();
    #endif
    #endif
        //ql_nw_app_init();
        //ql_datacall_app_init();
        //ql_osi_demo_init();
    #endif
    #ifdef QL_APP_FEATURE_SDMMC
        //ql_sdmmc_app_init();
    #endif

    #ifdef QL_APP_FEATURE_FILE
        //ql_fs_demo_init();
    #endif

```

Figure 3: SDMMC Demo ql_init_demo_thread()

3.2. SDMMC Debugging

3.2.1. Debugging Preparation

If you need to debug the SDMMC features, you can connect an external SD card to the Quectel LTE OPEN EVB, or use the eMMC chip on the LTE OPEN EVB.

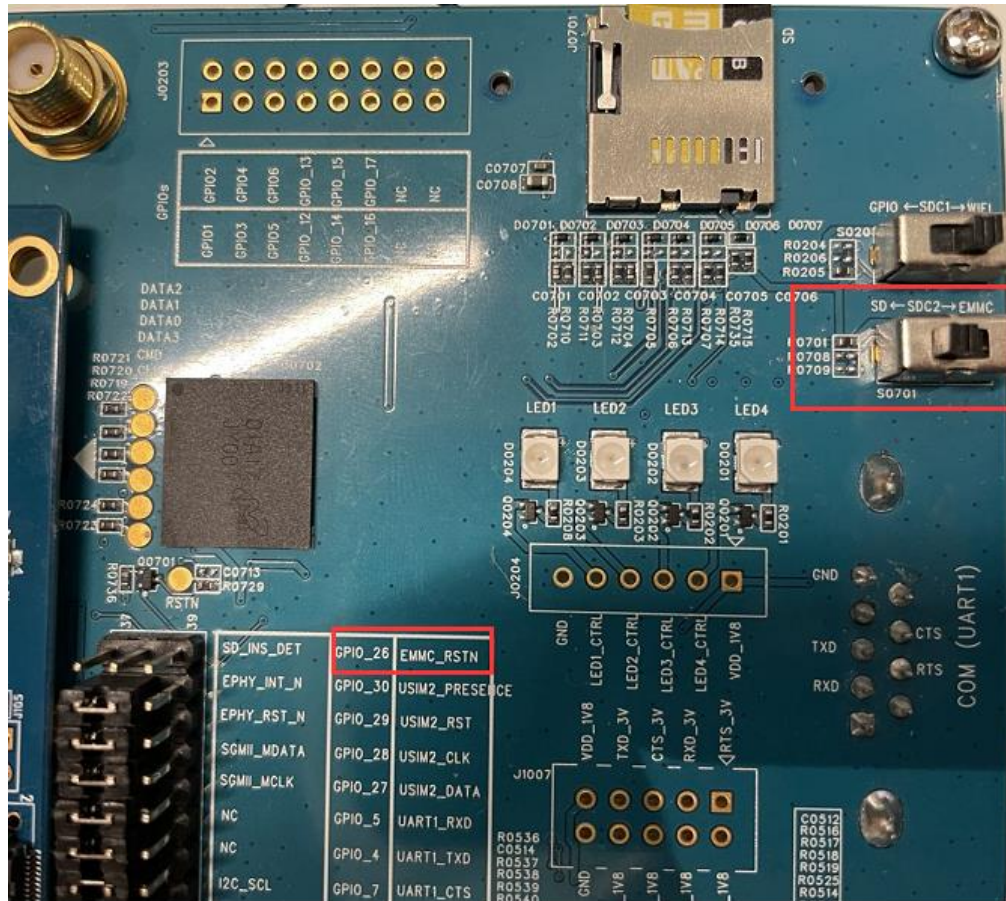


Figure 4: LTE OPEN EVB

When using SD card for debugging, flip the DIP switch in the upper right corner to the SD side and remove the jumper cap at GPIO_26. When using eMMC for debugging, flip the DIP switch in the upper right corner to the eMMC side and apply a short circuit between the jumper cap at GPIO_26 and the EMMC_RSTN side.

SDMMC function debugging steps are as follows:

1. Download the compiled firmware to the module.
2. Connect the USB port of the LTE OPEN EVB to the PC with a USB cable.
3. EC200U series and EG91xU series modules use cooltools to capture logs and USB AP Log Port to display the system debugging information, see **document [3]** for details.
4. You can view information of the example after capturing the AP logs with cooltools.

3.2.2. SDMMC Feature Debugging

Uncomment the codes of `ql_sdmmc_app_init()` after the module starts to enable auto-startup, and you can see the related information of SDMMC through the Log information.

The debugging information is shown below:

Index	Received	Tick	Level	Description
688	20:09:34.694	65379	QOPN/I	[ql_SDMMC][ql_sdmmc_mount_demo, 62] Mount succeed
770	20:09:37.679	48893	QOPN/I	[ql_SDMMC][ql_sdmmc_demo_thread, 93] block num:15499264
771	20:09:37.679	48966	QOPN/I	[ql_SDMMC][ql_sdmmc_demo_thread, 106] sdmmc read :1234567890abcdefg
773	20:09:37.679	48984	QOPN/I	[ql_SDMMC][ql_sdmmc_demo_thread, 114] sdmmc set clk :25000000
774	20:09:37.679	48984	QOPN/I	[ql_SDMMC][ql_sdmmc_demo_thread, 117] exit ql_sdmmc_demo_thread

Figure 5: AP Log Information

4 Appendix References

Table 4: Related Documents

Document Name
[1] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Quick_Start_Guide
[2] Quectel_EC200U&EG91xU_Series_QuecOpen_File_System_API_Reference_Manual
[3] Quectel_EC200U&EG91xU_Series_QuecOpen(SDK)_Log_Capture_Guide

Table 5: Terms and Abbreviations

Abbreviation	Description
AP	Application Processor
API	Application Programming Interface
App	Application
eMMC	Embedded Multimedia Card
FAT	File Allocation Table
EVB	Evaluation Board
GPIO	General-Purpose Input/Output
ID	Identifier
LTE	Long-Term Evolution
PC	Personal Computer
SD	Secure Digital Card
SDK	Software Development Kit
SDHC	Secure Digital High Capacity

SDSC	Secure Digital Standard Capacity
SDXC	Secure Digital Extended Capacity
SDMMC	Secure Digital and MultiMedia Card
USB	Universal Serial Bus
